

Сервис с удаленной проверкой прав

Участок: Базовые механизмы прав

Особенность сервисов с удаленной проверкой прав заключается в том, что они могут работать без использования базы данных. Для этого хранилище прав было перенесено из локальной БД в облако на сервис **Управление правами**, а информация о пользователях — на сервис **Пользователи**.

Таким образом, [микросервисный подход](#) позволяет работать с правами и, при этом, не копировать на сервис таблицы прав и пользователей из БД.

Добавление модулей в сервис

Для работы сервиса с удаленной проверкой прав в конфигурацию **s3srv** добавьте из [SBIS SDK](#) модули:

- **PermissionChecker** — модуль, в котором содержится логика так называемого триггера проверки прав. Другими словами, это `prehook` (точка) жизненного цикла запроса, в котором происходит обращение к логике "можно"/"нельзя".
- **PermissionCore** — модуль, в котором содержится код логики вычисления "можно"/"нельзя".
- **PermissionProviderRemote** — модуль, в котором содержится логика настроек прав удаленного сервиса. Он формирует данные по системным ролям, зонам доступа и их настройке и отправляет на сервис "Управление правами".

Для работы с [микросервисом пользователя](#) добавьте в конфигурацию зависимости от следующих модулей:

- **Crud** — набор базовых операций, используемых при работе с базами данных.
- **EventManager** — менеджер событий синхронизации.
- **UserService** — сервис, который хранит информацию о пользователях и их идентификационных данных.
- **UserServiceCloud** — облачная реализация интерфейса `IUserService`.

Настройка системы прав

Для корректной работы системы прав необходимо выполнить настройку:

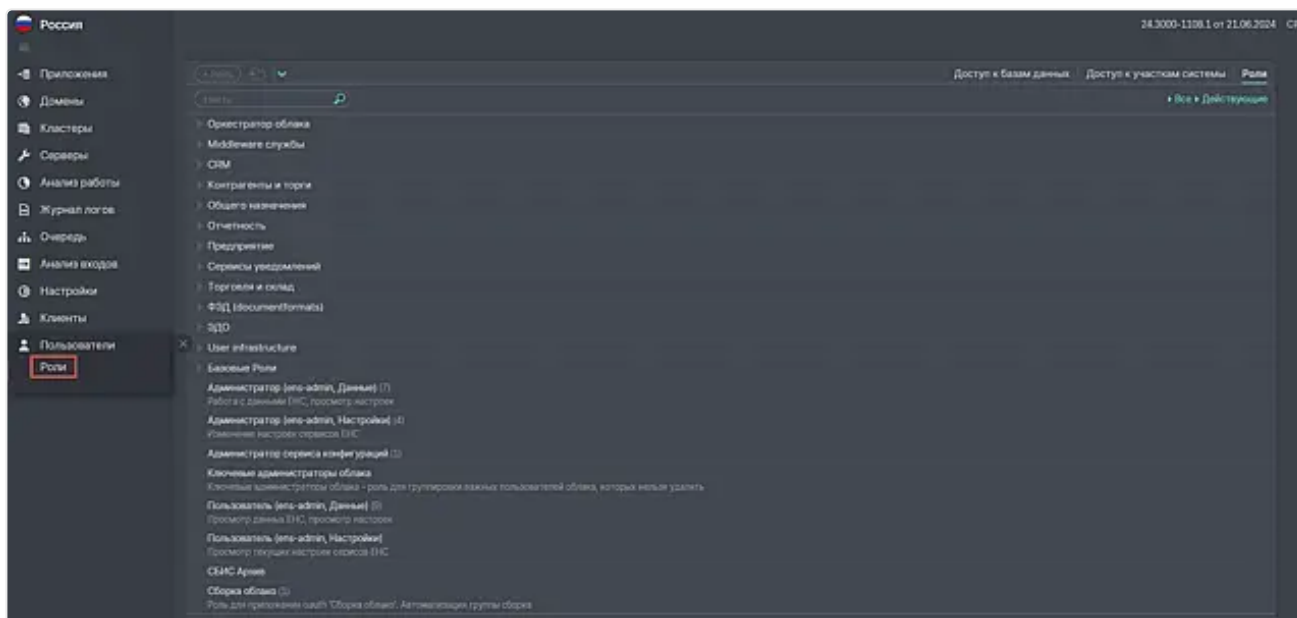
1. [Участков системы](#):

1. [Согласовать](#) создание участка системы.
2. [Создать](#) участок системы в справочнике `*.uax` в [Genie](#) и заполнить необходимые метаданные. При создании участков нет ограничений, они создаются в соответствии с функциональной областью.
3. Определить список разрешенных объектов и методов в участке системы.

2. [Ролей](#):

1. [Создать](#) роль в справочнике `*.rlx` в [Genie](#) и заполнить необходимые метаданные.
Как правило, создается одна административная роль (например, "Администратор сервиса DWC"), предоставляющая полные привилегии на управление сервисом. Созданная роль наследуется в роль "Администратор прикладных сервисов", в которой будут собраны права доступа на все сервисы облака системы прав.
2. Включить участки системы с требуемым разрешением в созданные роли.
3. Задать доступ по умолчанию для пользователей без ролей, настроив роль "Пользователь системы".

Все роли хранятся централизованно на сервисе "Управление правами", а посмотреть их можно через интерфейс сервиса "[Управление облаком](#)" в разделе **Пользователи** → **Роли**. Также с помощью интерфейса можно настроить доступ для пользователей.

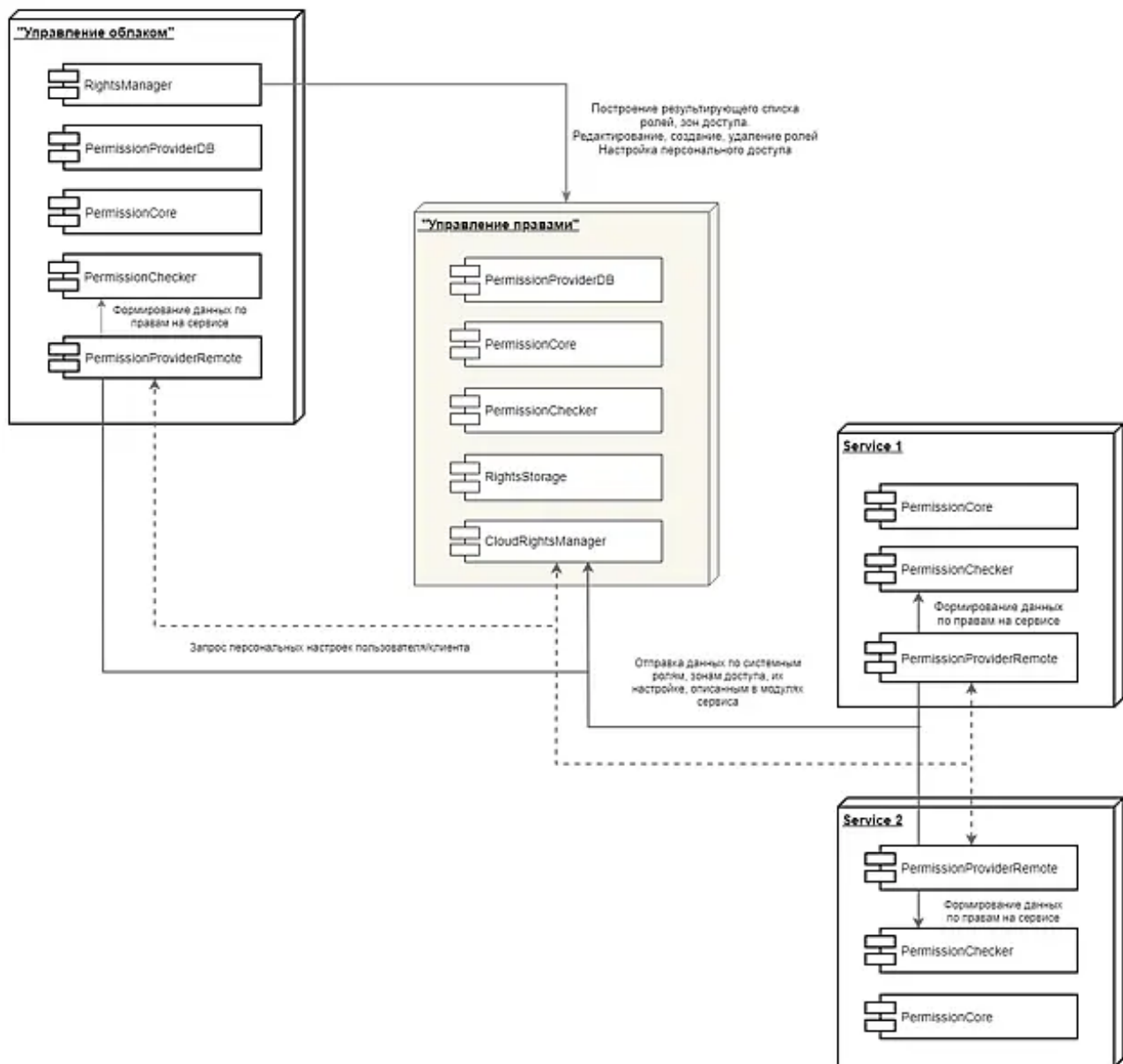


Старт сервиса и загрузка прав

При старте сервиса модуль **PermissionProviderRemote** формирует данные по системным ролям, участкам системы и их настройке и отправляет на сервис "Управление правами". Полученные сервисом данные попадают в единую базу с таблицами ролей и участков системы.

При первом обращении пользователя к сервису модуль **PermissionProviderRemote** снова обращается к сервису "Управление правами" и на основе полученных данных вычисляет права доступа для текущего пользователя. Таким образом, "Управление правами" позволяет просматривать, редактировать и централизованно хранить настройки ролей, а так же персональный доступ пользователей на сервисы облака. Сервис не имеет интерфейса, поэтому для отображения данных используется интерфейс сервиса "Управление облаком".

Ниже наглядно представлена схема взаимодействия сервисов.



Проверка прав доступа

Проверка прав методами

Система прав автоматически проверяет разрешение на исполнение любого внешнего вызова для текущего пользователя. Для реализации прикладных проверок на БЛ система прав предоставляет набор методов объекта [CheckRights](#), который включает в себя проверку доступа к участкам системы и методам:

AccessAreaRestrictions (AccessArea)	Возвращает все ограничения для зоны доступа.
AccessAreaRestrictions (Restrictions, AccessArea)	Возвращает конкретное ограничение для зоны доступа.
AccessAreas (AccessArea, Filter)	Возвращает способ доступа и ограничения для переданных участков системы.
AccessAreasJSON ()	Формирует json с разрешенными участками системы и ограничениями текущего пользователя.
AccessAreasJSON (Hash)	Формирует json-файл с разрешенными участками системы и ограничениями текущего пользователя.
AccessAreasRestrictions (AccessAreas)	Возвращает все ограничения для переданных зон доступа.
AccessAreasRestrictions (Restriction, AccessAreas)	Возвращает мерж настройки по области видимости для

	зон доступа.
ZoneAccess (Действие)	Возвращает уровень доступа к участку системы.
AccessAreaRestrictionsForUser (AccessArea, UserId)	Возвращает все ограничения для зоны доступа для указанного пользователя.
AccessAreaRestrictionsForUsers (ConstraintId, AccessAreaIds, UserIds)	Возвращает ограничения по массиву пользователей.
AccessAreasRestrictionsForUser (Restriction, AccessAreas, UserId)	Возвращает мерж настройки по области видимости для массива зон доступа для переданного идентификатора пользователя.
CheckAccessAreaForUser (UserId, ActionId)	Возвращает уровень доступа к участку системы для указанного пользователя.
CheckAccessAreasForUser (UserId, ActionIds)	Возвращает уровень доступа для массива участков системы по указанному пользователю.
AccessAreasList (Id, Flags, Restrictions)	Формирует RecordSet с разрешенными участками системы и ограничениями текущего пользователя.

При наличии интерфейса возможно добавить проверку прав на визуальные компоненты.

Проверка прав вручную

Чтобы убедиться в корректности настройки и работоспособности системы прав на вашем сервисе, необходимо выполнить следующие действия:

1. Выбрать метод БЛ, настроенный в пункте 1 [настройки системы прав](#).
2. В консоли выполнить запрос, проверяющий доступность данного метода:

- для пользователя с правами;
- для пользователя без прав. Пример запроса:

```

1 requirejs(['Types/source'], function(blo) {
2   new blo.SbisService({
3     endpoint: {
4       contract: 'ПроверкаПрав',
5       address: '/service/?x_version=20.6103-18'
6     }
7   }).call(
8     'ДоступностьМетода', {
9       "ИмяМетода": "<имя метода>",
10      "ЧислоАргументов": <количество аргументов>
11    }
12  ); // .addBoth(console.info);
13 });
```

3. Результат должен соответствовать настройкам доступа на метод.

- Для пользователя с правами будет возвращено значение true;
- Для пользователя без прав — false.

Встроенная проверка прав из online на стороннем сервисе

Для организации встроенной проверки прав необходимо:

1. Обеспечить наличие участка в метаданных online и прикладного сервиса. Для надежности выделить назначение прав (.iax- и .lix-файлы) в отдельный БЛ модуль и включить этот модуль в свой прикладной и в основной сервисы.
2. Включить в прикладной сервис модули для удаленной проверки прав — **PermissionProviderRemote**, **PermissionChecker** и **PermissionCore**.

Источник прав определяется классом пользователя. Если пользователь имеет класс "__сбис_администраторы" — настройки запрашиваются по текущему сервису, если принадлежит классу "__сбис_клиенты" — по основному сервису. Для проверки можно использовать все методы объекта CheckRights, за исключением массовых.



Проверка прав online на стороннем сервисе прав не поддерживает:

1. Ограничения, в том числе платформенные, например, "Удаление записей". Для получения значения ограничения нужно вручную делать запрос на online.
2. Возможность проверки наличия определенной роли у пользователя, поскольку оперирует итоговыми значениями уровня доступа к участкам системы.
3. Массовый расчет прав пользователей.